# Introduction to Artificial Intelligence
# Laboratory Work 2 – Variant 1

Tic-Tac-Toe with Minimax and Alpha-Beta Pruning

Anatoli Abahumna and Leonardo Meloni
Group 16

## 1 Introduction

### 1.1 Task Description

The goal of this laboratory work is to implement a **Tic-Tac-Toe** game in Python where a human plays against the computer. The board is a $3 \times 3$ matrix. Each cell contains either an empty space, `X`, or `O`. The first player always uses `X`, the second player always uses `O`, and the user can choose whether to play first or second.

The computer must play optimally using the **Minimax algorithm with alpha-beta pruning**. The program must also validate user input, print the board after each move, and correctly detect wins and draws.

### 1.2 Algorithm Overview

Minimax is a decision-making algorithm for two-player zero-sum games. One player tries to maximize the score, while the other tries to minimize it. In the implemented solution, the score is always measured from the **computer's perspective**:

- `+1` if the computer wins,
- `-1` if the human wins,
- `0` if the game ends in a draw.

Alpha-beta pruning improves Minimax by cutting off branches that cannot affect the final decision. It maintains two bounds:

- $\alpha$: the best value already guaranteed for the maximizing player,
- $\beta$: the best value already guaranteed for the minimizing player.

Whenever $\beta \leq \alpha$, the rest of that subtree can be skipped safely.

#### Advantages

- It guarantees optimal play.
- Alpha-beta pruning reduces the number of explored nodes.
- It is practical for small games such as Tic-Tac-Toe.

**Disadvantages**

- The search tree still grows exponentially with depth.
- For large games such as chess or checkers, complete search is not feasible.
- The amount of pruning depends on move ordering.

# 2 Implementation

The final solution is implemented in one class, `TicTacToe`, in the file `lab2_v1.py`. It is a merged version that keeps the required template structure and combines it with clearer console interaction.

## 2.1 Template Corrections

The provided template contained two issues:

- `print_board()` used `board` instead of `self.board`,
- the `__main__` block tried to call `play_tic_tac_toe()` without creating an instance of `TicTacToe`.

Both were fixed in the final implementation.

## 2.2 Board Representation and Helper Methods

The board is stored as a $3 \times 3$ list of characters. The helper methods `check_winner()` and `is_board_full()` are used by the game loop and by Minimax.

```
def check_winner(self, board=None):
    if board is None:
        board = self.board

    for row in board:
        if row[0] != EMPTY and row[0] == row[1] == row[2]:
            return row[0]

    for col in range(3):
        if board[0][col] != EMPTY and board[0][col] == board[1][col] ==
           board[2][col]:
            return board[0][col]

    if board[0][0] != EMPTY and board[0][0] == board[1][1] == board
       [2][2]:
        return board[0][0]

    if board[0][2] != EMPTY and board[0][2] == board[1][1] == board
       [2][0]:
        return board[0][2]

    return None
```

```python
def is_board_full(self, board=None):
    if board is None:
        board = self.board

    for row in board:
        if EMPTY in row:
            return False
    return True
```

## 2.3 Evaluation Function

There is no separate `evaluate()` method in the final version. Instead, terminal-state evaluation is performed directly at the beginning of `minimax()`:

```python
winner = self.check_winner(board)
if winner == self.computer_symbol:
    return 1
if winner == self.human_symbol:
    return -1
if self.is_board_full(board):
    return 0
```

This means the score is always interpreted from the computer's point of view. The `depth` parameter remains in the method signature because it was part of the template, but it is not needed for this evaluation scheme.

## 2.4 Minimax with Alpha-Beta Pruning

The method `minimax()` recursively tries all legal moves. When `maximizing_player` is true, the computer makes a move and the algorithm searches for the highest possible score. Otherwise, the human makes a move and the algorithm searches for the lowest possible score.

```python
def minimax(self, board, depth, maximizing_player, alpha, beta):
    winner = self.check_winner(board)
    if winner == self.computer_symbol:
        return 1
    if winner == self.human_symbol:
        return -1
    if self.is_board_full(board):
        return 0

    if maximizing_player:
        best_value = float("-inf")
        for row in range(3):
            for col in range(3):
                if board[row][col] == EMPTY:
                    board[row][col] = self.computer_symbol
                    value = self.minimax(board, depth + 1, False, alpha,
                        beta)
                    board[row][col] = EMPTY
                    best_value = max(best_value, value)
                    alpha = max(alpha, best_value)
                    if beta <= alpha:
                        return best_value
```

```
        return best_value

    best_value = float("inf")
    for row in range(3):
        for col in range(3):
            if board[row][col] == EMPTY:
                board[row][col] = self.human_symbol
                value = self.minimax(board, depth + 1, True, alpha, beta
                    )
                board[row][col] = EMPTY
                best_value = min(best_value, value)
                beta = min(beta, best_value)
                if beta <= alpha:
                    return best_value
    return best_value
```

Important implementation details:

- Every move is written to the board before the recursive call and undone immediately after it.
- The search order is row-major: $(0,0), (0,1), (0,2), (1,0), \ldots, (2,2)$.
- Pruning happens when $\beta \leq \alpha$.

## 2.5 Best Move Selection

The method `best_move()` checks every empty cell, evaluates it with `minimax()`, and returns the move with the highest score:

```
def best_move(self):
    best_score = float("-inf")
    move = None

    for row in range(3):
        for col in range(3):
            if self.board[row][col] == EMPTY:
                self.board[row][col] = self.computer_symbol
                score = self.minimax(self.board, 0, False,
                                     float("-inf"), float("inf"))
                self.board[row][col] = EMPTY

                if score > best_score:
                    best_score = score
                    move = (row, col)

    return move
```

Since the score is always defined from the computer's perspective, the computer always chooses the move with the largest returned value.

## 2.6 Input Validation and Game Loop

The game loop asks whether the user wants to play first or second, prints instructions, validates the coordinates, prevents moves in occupied cells, and prints the board after each move.

```
while True:
    try:
        row = int(input("Enter row (0-2): "))
        col = int(input("Enter column (0-2): "))
    except ValueError:
        print("Invalid input. Please enter integers from 0 to 2.")
        continue

    if row < 0 or row > 2 or col < 0 or col > 2:
        print("Coordinates out of range. Please enter values from 0 to
            2.")
        continue

    if self.board[row][col] != EMPTY:
        print("That cell is already occupied. Choose another one.")
        continue

    self.board[row][col] = self.human_symbol
    break
```

This guarantees that the program does not crash on non-numeric input and does not accept invalid coordinates or illegal moves.

# 3 Discussion

## 3.1 Verification of the Final Program

Before writing the report, the final merged implementation was checked in three ways:

- it was compiled with Python syntax checking,
- it was tested exhaustively to confirm that the human cannot force a win when the computer plays either first or second,
- the two decision-tree examples below were checked against the exact row-major search order used by the implementation.

Therefore, the claims about the algorithm and the two decision trees below are based on the verified behavior of the final program.

## 3.2 Alpha-Beta Pruning: Test Cases

The assignment asks for additional examples with exactly three empty cells left. In such positions, the root has three legal moves, so the tree starts with three branches.

For the two examples below, `X` is taken as the maximizing player and `O` as the minimizing player. This matches the example style from the assignment and the generated images. The values and pruned branches were checked against the implemented search order.

### 3.2.1 Test Case 1

**Initial board** (`X` to move, MAX):

```
X | O | O
---------
  |   | X
---------
  | X | O
```

Empty cells: $(1,0)$, $(1,1)$, $(2,0)$.

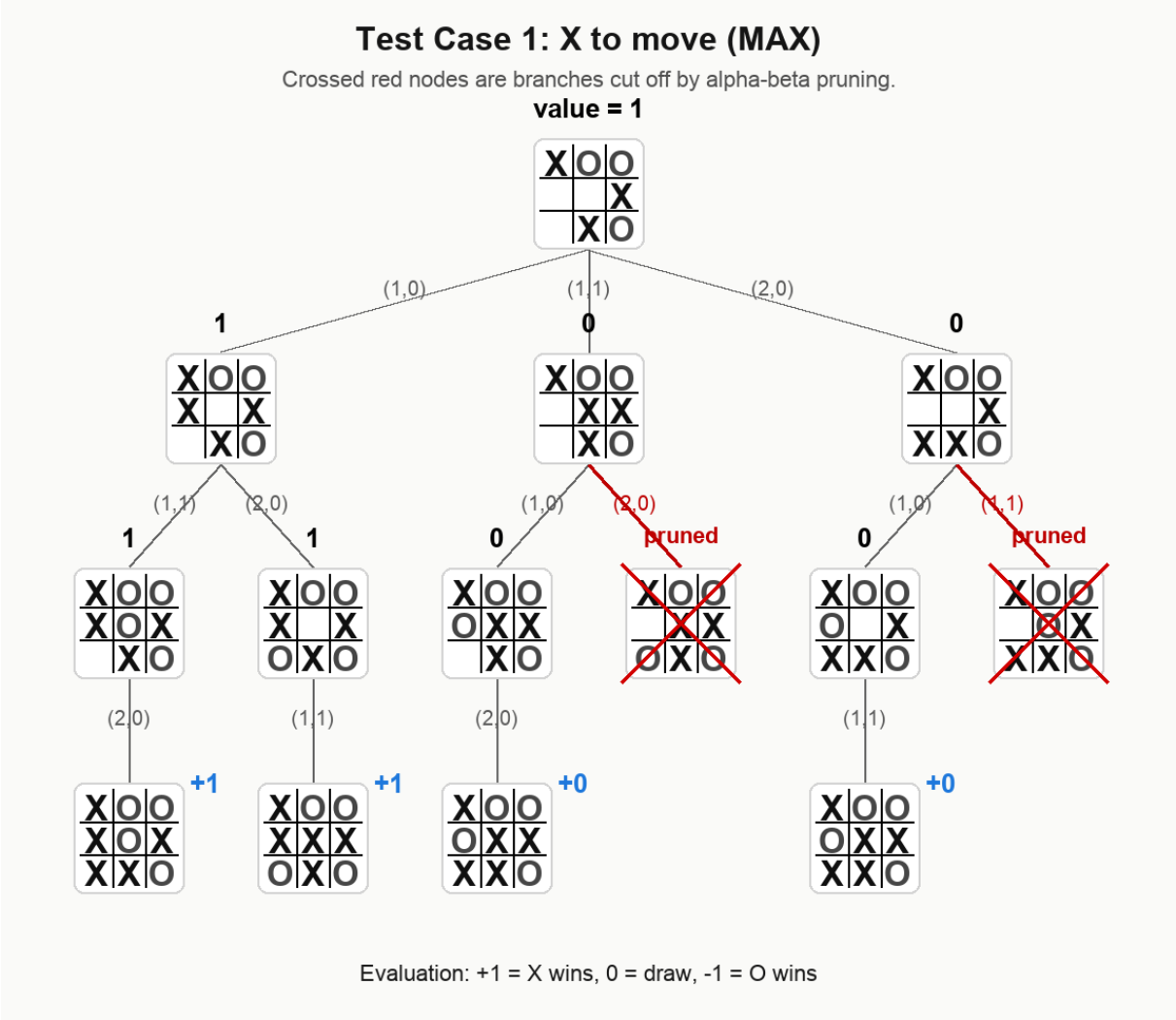



Figure 1: Decision tree for Test Case 1. Crossed red nodes are pruned by alpha-beta pruning.

The verified first-level branch values are:

$$(1,0) \to 1, \qquad (1,1) \to 0, \qquad (2,0) \to 0.$$

So the root value is:

$$\max(1,0,0) = 1.$$

This means `X` has a forced win from this position.
The verified pruned branches are:

- under the branch $(1, 1)$, the child $(2, 0)$ is pruned,
- under the branch $(2, 0)$, the child $(1, 1)$ is pruned.

These cutoffs occur because after the first branch is evaluated, the root already has $\alpha = 1$. In the later minimizing branches, as soon as MIN finds a continuation with value 0, the condition $\beta \leq \alpha$ holds and the remaining sibling branch is skipped.

### 3.2.2 Test Case 2

**Initial board** (`X` to move, MAX):

```
X | X | O
---------
O |   | X
---------
  | O |
```

Empty cells: $(1, 1)$, $(2, 0)$, $(2, 2)$.

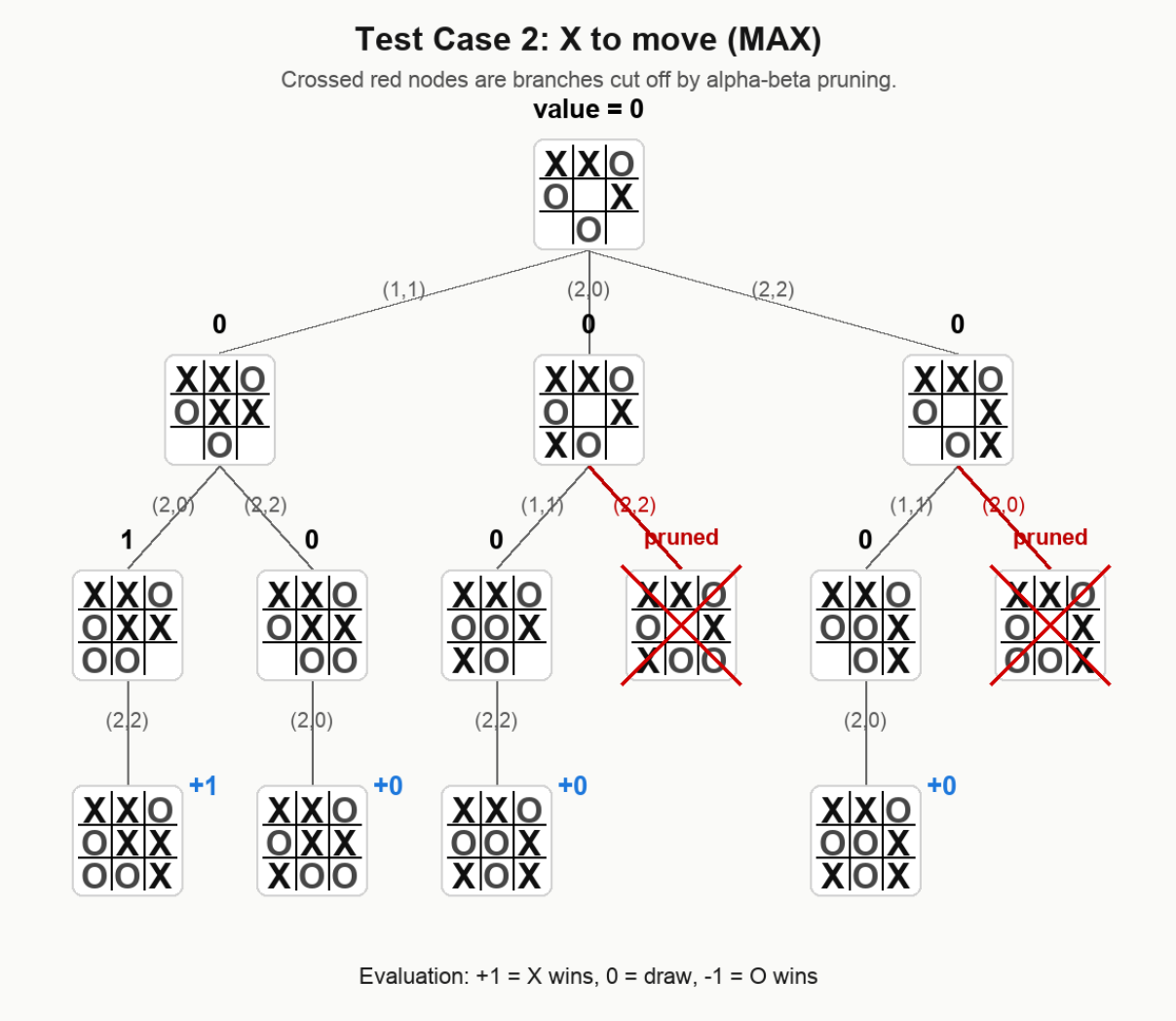



Figure 2: Decision tree for Test Case 2. Crossed red nodes are pruned by alpha-beta pruning.

The verified first-level branch values are:

$$(1,1) \to 0, \qquad (2,0) \to 0, \qquad (2,2) \to 0.$$

So the root value is:

$$\max(0,0,0) = 0.$$

Therefore, with optimal play, the game ends in a draw.
The verified pruned branches are:

- under the branch $(2,0)$, the child $(2,2)$ is pruned,
- under the branch $(2,2)$, the child $(2,0)$ is pruned.

Here the first branch already establishes $\alpha = 0$. In the later minimizing branches, once MIN finds a line with value 0, the remaining sibling cannot improve the result for MAX, so it is pruned.

## 3.3 Evaluation Function Analysis

The evaluation function used by the program is:

$$f(\text{board}) = \begin{cases} +1 & \text{if the computer wins,} \\ -1 & \text{if the human wins,} \\ 0 & \text{if the board is full and nobody wins.} \end{cases}$$

This is a **terminal-state evaluation**. It does not estimate the value of non-terminal positions. Instead, the algorithm searches to terminal states and then propagates the exact result back through the tree.

This works very well for Tic-Tac-Toe because the game is small enough for exhaustive search. As a result, the program can always choose an optimal move and should never lose.

## 3.4 Would This Work for Checkers or Chess?

This evaluation function would **not** work well for checkers or chess.

The main reason is that those games are far too large to search to the end in most positions. If we stopped the search early but still used only the values `+1`, `-1`, and `0`, then most non-terminal positions would look identical to the algorithm. That would give the search almost no useful guidance.

For larger games, a **heuristic evaluation function** is necessary. Such a function estimates how good a position is even if the game is not over. For example:

- in chess, the evaluation may use material balance, king safety, pawn structure, mobility, and control of important squares,
- in checkers, the evaluation may use number of pieces, number of kings, mobility, and positional advantage.

So, the evaluation function from this laboratory is appropriate for Tic-Tac-Toe, but it is not sufficient for more complex games unless it is replaced by a stronger heuristic one.

# 4 Conclusion

In this laboratory work, a complete Tic-Tac-Toe game was implemented using Minimax with alpha-beta pruning. The final program allows the user to choose whether to play first or second, validates all inputs, prints the board after each move, and makes optimal moves for the computer.

The final verified implementation shows that:

- Minimax can solve Tic-Tac-Toe exactly,
- alpha-beta pruning reduces unnecessary search,
- a terminal-state evaluation is sufficient for Tic-Tac-Toe,
- more complex games such as checkers and chess require heuristic evaluation functions.

The two decision-tree examples also show an important practical fact: the exact pruned branches depend on move ordering. In this report, the values and crossed-out branches were matched to the actual row-major search order used in the final program.